CAMTIC ST-mini
자율주행 로봇 사용 매뉴얼

접속 · 실행 · 상태 확인 · 개발 안내

문서명	CAMTIC ST-mini 자율주행 로봇 사용 매뉴얼
버전	2.0
작성일	2026-11-05
작성	넥스루트 주식회사
대상	캠틱 전국홍 선임님

목차

1. 이 문서를 보는 방법
2. 로봇 구성
3. 접속하기
4. 전원 켜기 · 끄기
5. 로봇 실행하기
6. 상태 확인
7. CAN 자동 복구 기능
8. 개발하기
9. 문제 해결
10. 명령어 요약
11. 주의사항

1. 이 문서를 보는 방법

1-1. 명령어 실행 위치 표시

이 문서의 명령어에는 실행하는 곳이 표시되어 있습니다.

표시	실행하는 곳	터미널 모습 (프롬프트)
내 PC	로봇에 접속하는 담당자의 PC	(사용하는 PC에 따라 다름)
camtic_dev 안	로봇 안의 개발용 공간 (ROS 2 사용 가능)	`root@camtic:/home/camtic#`
로봇 PC	로봇의 기본 터미널 (전원 · 서비스 · 네트워크 관리)	`camtic@camtic:~$`

> 로봇에 접속하면 **자동으로 camtic_dev 안으로 들어갑니다.** 로봇 PC 명령은 `exit` 를 한 번 입력해 나온 뒤 실행합니다. 프롬프트가 `root@` 로 시작하면 camtic_dev 안, `camtic@` 로 시작하면 로봇 PC입니다.

1-2. 기호

기호	의미
`<와이파이 이름>` 처럼 꺾쇠로 감싼 부분	실제 값으로 바꿔 입력합니다 (꺾쇠는 입력하지 않음)
`Ctrl + C`	실행 중인 프로그램을 끝냅니다
`#` 뒤의 글	설명입니다. 입력하지 않아도 됩니다

2. 로봇 구성

2-1. 장치

장치	역할	연결
로봇 PC (Jetson AGX Orin)	전체 제어, 자율주행 계산	-
Scout Mini 베이스	바퀴 구동, 배터리	CAN (`can0`)
RPLidar S3	주변 거리 측정 (라이다)	USB (`/dev/rplidar`)
MW-AHRS	자세 · 회전 측정 (IMU)	USB (`/dev/mw_ahrs`)
RealSense	영상 · 깊이 측정 (카메라)	USB

항목	내용
로봇 PC 운영체제	Ubuntu 20.04 (JetPack 5.1.3)
로봇 소프트웨어	ROS 2 Humble (Docker 컨테이너 안에서 실행)
계정	`camtic`

2-2. 소프트웨어 구성

로봇 PC에는 ROS가 직접 설치되어 있지 않습니다. ROS 2 Humble은 **Docker 컨테이너**라는 독립된 공간 안에서 실행됩니다. 컨테이너는 용도별로 4개입니다.

컨테이너	용도	평소 상태
`camtic_dev`	개발 · 시험용. 직접 들어가서 명령을 실행하는 공간	**항상 켜져 있음** (대기)
`camtic_nav`	자율주행 전체 (운영 모드)	꺼져 있음
`camtic_zenoh`	관제 서버와 통신 (운영 모드)	꺼져 있음
`camtic_slam`	지도 작성	꺼져 있음

- 모든 컨테이너는 로봇의 작업 폴더(`/home/camtic/camtic_ws`)와 홈 폴더(`/home/camtic`)를 함께 사용합니다.

3. 접속하기

3-1. 접속 방법

방법	주소	특징
랜선 직결	`192.168.1.211` (고정)	주소가 항상 같아 별도 확인 없이 바로 접속할 수 있습니다
와이파이	공유기가 정해 준 주소 (3-3에서 확인)	선 없이 접속. 주소가 바뀔 수 있습니다
화면 접속 (NoMachine)	위 두 주소 중 하나	로봇 화면을 그대로 봅니다
모니터 · 키보드 직접 연결	-	네트워크 설정이 안 될 때 사용합니다

3-2. 랜선 직결

로봇의 유선 포트는 `192.168.1.211`로 고정되어 있습니다. 내 PC의 유선 주소를 같은 대역(`192.168.1.x`)으로 한 번만 설정하면 됩니다.

내 PC가 Ubuntu인 경우

```
# 내 PC
ip -br link                    # 랜카드 이름 확인 (예: enp52s0, eth0)

# 고정 주소 설정 (처음 한 번)
nmcli con add type ethernet ifname <랜카드 이름> con-name robot-lan \
    ipv4.method manual ipv4.addresses 192.168.1.21/24 ipv4.never-default yes

# 연결 · 접속
nmcli con up robot-lan
ssh camtic@192.168.1.211
```

내 PC가 Windows인 경우

1. `설정` → `네트워크 및 인터넷` → `이더넷` → `IP 할당` 옆 `편집`
2. `수동` 선택 → `IPv4` 켜기
3. IP 주소 `192.168.1.21`, 서브넷 마스크 `255.255.255.0` 입력 (게이트웨이는 비워 둠) → 저장

4. PowerShell을 열고 접속합니다.

```
# 내 PC
ssh camtic@192.168.1.211
```

> 내 PC 주소는 `192.168.1.211`만 아니면 `192.168.1.2` ~ `192.168.1.254` 중 아무 번호나 사용할 수 있습니다.

3-3. 와이파이로 접속

로봇의 와이파이 주소 확인 (랜선 직결이나 모니터로 로봇에 들어간 상태에서)

```
# 로봇 PC
ip -br addr show wlan0        # 예: wlan0  UP  192.168.0.157/24  → 주소는 192.168.0.157
```

로봇을 새 와이파이에 연결 (현장 와이파이를 처음 쓸 때)

```
# 로봇 PC
sudo nmcli dev wifi list                                          # 주변 와이파이 목록
sudo nmcli dev wifi connect "<와이파이 이름>" password "<비밀번호>"
ip -br addr show wlan0                                            # 새 주소 확인
```

- 한 번 연결한 와이파이는 다음부터 자동으로 연결됩니다.
- 공유기는 로봇 주소를 바꿀 수 있습니다. 매번 같은 주소로 접속하려면 공유기 설정에서 로봇 주소를 고정하는 것을 권장합니다.

접속

```
# 내 PC
ssh camtic@<로봇 와이파이 주소>
```

3-4. 화면 접속 (NoMachine)

1. 내 PC에 NoMachine을 설치합니다 (nomachine.com에서 무료 다운로드).
2. NoMachine에서 `Add` → Host에 로봇 주소(`192.168.1.211` 또는 와이파이 주소) 입력
3. 계정 `camtic`과 비밀번호로 접속합니다.

3-5. 로봇 고정 주소 확인 · 변경

```
# 로봇 PC
nmcli -g ipv4.addresses con show Backup                          # 확인 (Backup = 유선 설정 이름)
sudo nmcli con mod Backup ipv4.addresses 192.168.1.211/24        # 변경할 때
sudo nmcli con up Backup                                         # 적용 (랜선 연결 상태에서)
```

3-6. 접속하면 보이는 화면

```
root@camtic:/home/camtic#        ← 자동으로 camtic_dev 안에 들어온 상태
```

- 여기서 ROS 2 명령(`ros2 ...`)을 바로 사용할 수 있습니다.
- `exit`를 입력하면 로봇 PC(`camtic@camtic:~$`)로 나오고, 한 번 더 `exit`를 입력하면 접속이 끝납니다.
- camtic_dev가 꺼져 있을 때는 로봇 PC로 바로 접속됩니다.

4. 전원 켜기 · 끄기

4-1. 전원을 켜면 자동으로 되는 것

항목	자동 여부
Docker, `camtic_dev` 컨테이너 (대기 상태)	**자동으로 켜짐**
CAN(`can0`) 설정 · CAN 자동 복구 기능 (7장)	**자동으로 켜짐**
Scout 드라이버, 센서, 자율주행	켜지지 않음 → **5장에 따라 실행**

4-2. 끄기

1. 실행 중인 프로그램을 `Ctrl + C`로 끝냅니다.
2. 로봇 PC에서 끕니다.

```
# 로봇 PC
sudo poweroff
```

- 전원 버튼을 길게 눌러 강제로 끄는 것은 피합니다. 저장 중인 파일이 손상될 수 있습니다.

5. 로봇 실행하기

5-1. 두 가지 실행 방식

구분	개발 모드	운영 모드
용도	시험 · 개발 · 확인	관제 서버와 연동한 현장 운영
실행 방법	camtic_dev 안에서 명령 입력	로봇 PC에서 명령 한 줄
결과 확인	실행한 터미널에 바로 표시	로그 명령으로 확인
전원을 다시 켜면	다시 실행해야 함	자동 실행 설정 가능

> **두 방식을 동시에 실행하지 않습니다.** 바퀴 · 센서 프로그램이 두 번 실행되어 서로 충돌합니다.

5-2. 개발 모드 (기본)

1) 접속 — 3장의 방법으로 접속하면 camtic_dev 안으로 들어옵니다.

2) 상태 확인

```
# camtic_dev 안
ip -br link show can0          # UP 이면 정상
pgrep -x scout_base_node       # 숫자가 나오면 바퀴 프로그램이 이미 실행 중 (7장 자동 복구가 띄운 것)
pkill -x scout_base_node       # 위에서 숫자가 나온 경우에만 실행: 끄고 시작
```

3) 실행

```
# camtic_dev 안
ros2 launch camtic_bringup camtic_bringup.launch.py     # 바퀴 + 센서 + 자율주행 + 위치 복원
```

4) 확인 — 새 터미널을 열어 접속합니다 (자동으로 camtic_dev 안).

```
# camtic_dev 안
ros2 topic hz /wheel_odom      # 약 50Hz 이면 바퀴 정상
ros2 topic hz /scan            # 숫자가 나오면 라이다 정상
```

5) 끄기 — 3)을 실행한 터미널에서 `Ctrl + C`

5-3. 운영 모드 (관제 서버 연동)

```
# 로봇 PC
sudo systemctl start camtic-robot-nav camtic-robot-zenoh      # 켜기
docker ps                                                     # 확인 (1분 뒤 두 컨테이너가 healthy)
sudo systemctl stop camtic-robot-nav camtic-robot-zenoh       # 끄기
```

전원을 켤 때마다 자동으로 운영 모드가 실행되게 하려면:

```
# 로봇 PC
sudo systemctl enable --now camtic-robot-nav camtic-robot-zenoh     # 자동 실행 켜기
sudo systemctl disable --now camtic-robot-nav camtic-robot-zenoh    # 자동 실행 끄기
```

- 운영 모드는 관제 서버가 준비되어 있어야 합니다. 관제 서버 주소 변경은 넥스루트 주식회사에 문의합니다.
- 운영 모드는 위의 `systemctl` 명령으로만 켜고 끕니다. `docker compose`로 직접 켜면 컨테이너가 다시 만들어집니다.

5-4. 일부만 실행하기 (개발 모드)

기능 단위 — 순서: 바퀴 → 센서 → 자율주행 → 위치 복원

대상	명령어
바퀴 (Scout + 배터리)	`ros2 launch camtic_bringup base.launch.py`
센서 전체 (라이다, 카메라, IMU)	`ros2 launch camtic_bringup sensors.launch.py`
자율주행 (지도, 위치 추정, 경로)	`ros2 launch camtic_bringup navigation.launch.py`
위치 복원	`ros2 launch camtic_bringup manager.launch.py`
지도 작성 (바퀴 + 센서 + SLAM)	`ros2 launch camtic_bringup slam_bringup.launch.py`

장치 단위

대상	명령어
바퀴만	`ros2 launch scout_base scout_mini_base.launch.py`
라이다만	`ros2 launch rplidar_ros rplidar_s3_launch.py serial_port:=/dev/rplidar`
IMU만	`ros2 run mw_ahrs_ros2 mw_ahrs_driver_node --ros-args -p port:=/dev/mw_ahrs -p frame_id:=imu_link`

대상	명령어
카메라만	`ros2 launch realsense2_camera rs_launch.py pointcloud.enable:=true`

- 모두 **camtic_dev 안**에서 실행합니다. 하나씩 실행할 때는 터미널을 여러 개 엽니다.
- 자율주행은 바퀴와 센서가 먼저 실행되어 있어야 동작합니다.
- 자율주행이 사용하는 기본 지도는 `/home/camtic/camtic_ws/src/camtic_bringup/maps/map.yaml` 입니다.

5-5. 끄기

대상	방법	실행 위치
터미널에서 실행한 프로그램	`Ctrl + C`	실행한 터미널
자동 복구가 띄운 바퀴 프로그램	`pkill -x scout_base_node`	camtic_dev 안
운영 모드	`sudo systemctl stop camtic-robot-nav camtic-robot-zenoh`	로봇 PC

6. 상태 확인

6-1. 정상 동작 확인

확인 항목	명령어	정상	실행 위치
CAN 연결	`ip -br link show can0`	`UP`	어디서나
바퀴 데이터	`ros2 topic hz /wheel_odom`	약 50Hz	camtic_dev 안
베이스 상태	`ros2 topic echo --once /scout_status`	`error_code: 0`, 배터리 전압 표시	camtic_dev 안
라이다	`ros2 topic hz /scan`	숫자가 계속 나옴	camtic_dev 안
실행 중인 프로그램	`ros2 node list`	목록 표시	camtic_dev 안
컨테이너	`docker ps`	사용 중인 컨테이너가 `Up`	로봇 PC

6-2. 기록(로그) 보기

보고 싶은 것	명령어	실행 위치
개발 모드	실행한 터미널에 바로 표시	-
운영 모드 자율주행	`cd /home/camtic/camtic_ws && docker compose logs -f camtic-nav`	로봇 PC
운영 모드 관제 통신	`cd /home/camtic/camtic_ws && docker compose logs -f camtic-zenoh`	로봇 PC
CAN 자동 복구 기록	`journalctl -t can0-watchdog -f`	로봇 PC
자동 복구가 띄운 바퀴 프로그램	`tail -f /home/camtic/camtic_ws/log/scout_auto.log`	어디서나

- `-f`가 붙은 명령은 기록을 계속 보여 줍니다. `Ctrl + C`로 빠져나옵니다.

7. CAN 자동 복구 기능

7-1. 하는 일

바퀴(Scout 베이스)는 CAN(`can0`)으로 연결되어 있습니다. USB 연결이 순간적으로 끊기면 CAN이 꺼지고 바퀴 프로그램도 멈춥니다. 이를 위해 로봇에는 자동 복구 기능이 켜져 있습니다.

상황	자동으로 하는 일
CAN이 꺼짐	2초 안에 다시 켭니다
CAN이 다시 켜짐	바퀴 프로그램을 camtic_dev 안에서 다시 실행합니다 (실패 시 3번까지 재시도)

7-2. 알아 둘 점

- **camtic_dev를 다시 시작한 직후에는 바퀴 프로그램이 자동으로 실행되지 않습니다.** 다음 명령으로 복구 기능을 이용해 실행할 수 있습니다 (약 20초).

  ```
  # 로봇 PC
  sudo ip link set can0 down
  ```

- 직접 실행한 바퀴 프로그램도, CAN이 끊기면 복구 기능이 끄고 다시 실행합니다. 이때 다시 실행된 프로그램은 터미널이 아닌 배경에서 동작합니다 (기록: 6-2).
- 자동 복구 끄기 · 켜기:

  ```
  # 로봇 PC
  sudo systemctl disable --now can0-watchdog    # 끄기
  sudo systemctl enable --now can0-watchdog     # 켜기
  ```

7-3. CAN이 자주 끊길 때

- USB-CAN 어댑터를 USB 허브가 아닌 **로봇 PC 본체 USB 포트에 직접** 연결합니다.
- 전원 공급형 USB 허브를 사용하고, 케이블과 연결 부위를 점검합니다.
- 계속되면 넥스루트 주식회사에 문의합니다.

8. 개발하기

8-1. 개발 환경 구조

구분	로봇 PC	camtic_dev 안
운영체제	Ubuntu 20.04	Ubuntu 22.04
ROS	없음	ROS 2 Humble
계정	`camtic`	`root` (sudo 불필요)
주로 하는 일	파일 편집, 전원 · 서비스 관리	ROS 실행, 빌드

- 로봇 PC의 `/home/camtic` 폴더는 camtic_dev 안에서도 **같은 경로**로 보입니다. 로봇 PC에서 편집한 파일을 camtic_dev 안에서 바로 빌드할 수 있습니다.
- camtic_dev 안에서 `~`는 `/root`입니다. 작업 폴더는 반드시 `/home/camtic/...` 경로로 찾습니다.

유지되는 것과 사라지는 것

구분	컨테이너 재시작 · 로봇 재부팅	컨테이너를 새로 만들 때
`/home/camtic` 아래 파일 (작업 폴더 포함)	유지	**유지**
camtic_dev 안에서 `apt install`, `pip install` 한 것	유지	**사라짐**

8-2. 작업 폴더 원칙

> **새로 개발하는 패키지는 `/home/camtic` 아래에 새 작업 폴더를 만들어 사용합니다.** 기존 작업 폴더 `camtic_ws`는 운영 모드 · 자동 복구 기능이 그대로 사용합니다. 이 안에 패키지를 추가하거나 다시 빌드하면 기존 기능이 동작하지 않을 수 있습니다.

8-3. 새 작업 폴더 만들기 · 소스 받기 · 빌드

```
# camtic_dev 안
# 1) 새 작업 폴더 만들기 (처음 한 번, 이름은 자유. 예: my_ws)
mkdir -p /home/camtic/my_ws/src

# 2) 소스 받기 (ROS 2 Humble용 브랜치 · 버전을 -b로 지정)
cd /home/camtic/my_ws/src
git clone -b <브랜치 또는 버전> <저장소 주소>

# 3) 필요한 패키지 자동 설치
cd /home/camtic/my_ws
rosdep update
rosdep install --from-paths src --ignore-src -r -y

# 4) 빌드 · 적용
source /root/camtic_ws/install/setup.bash     # 기존 camtic 패키지와 함께 쓸 때 (빌드 전에)
colcon build
source install/setup.bash
```

- 새 패키지를 직접 만들 때: `cd /home/camtic/my_ws/src && ros2 pkg create --build-type ament_python <패키지 이름>`
- camtic_dev 안에서 만든 파일은 소유자가 root가 됩니다. 로봇 PC에서 파일을 고칠 수 없으면:

  ```
  # 로봇 PC
  sudo chown -R camtic:camtic /home/camtic/my_ws
  ```

8-4. 프로그램(패키지) 설치

시험용 설치 — camtic_dev를 새로 만들면 사라집니다.

```
# camtic_dev 안
apt update
apt install -y ros-humble-<패키지 이름>     # 예: ros-humble-teleop-twist-keyboard
pip3 install <파이썬 패키지 이름>
```

계속 쓸 패키지 — 컨테이너 설계도(이미지)에 추가합니다.

```
# 로봇 PC
cd /home/camtic/camtic_ws
nano requirements-apt.txt          # apt 패키지 이름을 한 줄에 하나씩 추가
nano requirements.txt              # 파이썬 패키지 이름을 한 줄에 하나씩 추가
docker compose build               # 이미지 다시 만들기 (시간이 걸립니다)
docker compose up -d camtic-dev    # 새 이미지로 camtic_dev 다시 만들기
```

- camtic_dev를 다시 만든 뒤에는 바퀴 프로그램을 다시 실행합니다 (7-2).
- 로봇 PC(Ubuntu 20.04)에는 ROS 2 Humble 패키지를 설치할 수 없습니다. ROS 패키지는 항상 camtic_dev 안에 설치합니다.

9. 문제 해결

증상	확인	해결
랜선으로 접속이 안 됨	내 PC 주소가 `192.168.1.x`인지, `ping 192.168.1.211` 응답	3-2 설정 확인, 랜선 재연결
와이파이로 접속이 안 됨	로봇 와이파이 주소가 바뀌었는지	랜선 직결로 접속해 3-3으로 주소 확인
`/wheel_odom`이 안 나옴	`pgrep -x scout_base_node`	바퀴 프로그램 실행 (5-4) 또는 `sudo ip link set can0 down` (7-2)
`can0`가 `DOWN`	`journalctl -t can0-watchdog -n 20`	자동 복구 기능이 켜져 있는지 확인 (7-2), 베이스 전원 확인
바퀴 프로그램이 켜자마자 꺼짐	실행한 터미널의 마지막 메시지	이미 실행 중인 바퀴 프로그램이 없는지 확인 후 다시 실행
`ros2` 명령이 없다고 나옴	프롬프트가 `camtic@`인지	로봇 PC에 있는 상태 → `docker exec -it camtic_dev bash`로 들어감
`systemctl` · `docker` 명령이 안 됨	프롬프트가 `root@`인지	camtic_dev 안에 있는 상태 → `exit`로 나와서 실행
`no configuration file provided`	현재 폴더	`cd /home/camtic/camtic_ws` 후 다시 실행
만든 폴더가 camtic_dev 안에서 안 보임	찾은 경로	`~`가 아닌 `/home/camtic/<폴더 이름>`으로 이동
운영 모드 컨테이너가 `unhealthy`	`docker ps`	바퀴 연결 문제 → CAN · 베이스 전원 확인

10. 명령어 요약

접속

하고 싶은 것	명령어	위치
랜선으로 접속	`ssh camtic@192.168.1.211`	내 PC
와이파이로 접속	`ssh camtic@<로봇 와이파이 주소>`	내 PC
camtic_dev에서 로봇 PC로 나가기	`exit`	camtic_dev 안
로봇 PC에서 camtic_dev로 들어가기	`docker exec -it camtic_dev bash`	로봇 PC
로봇 와이파이 주소 확인	`ip -br addr show wlan0`	로봇 PC
로봇을 새 와이파이에 연결	`sudo nmcli dev wifi connect "<이름>" password "<비밀번호>"`	로봇 PC

실행 · 끄기

하고 싶은 것	명령어	위치
전체 실행 (개발 모드)	`ros2 launch camtic_bringup camtic_bringup.launch.py`	camtic_dev 안
바퀴만 실행	`ros2 launch scout_base scout_mini_base.launch.py`	camtic_dev 안
실행 중인 프로그램 끄기	`Ctrl + C`	실행한 터미널
자동 복구가 띄운 바퀴 프로그램 끄기	`pkill -x scout_base_node`	camtic_dev 안
바퀴 프로그램 자동 재실행	`sudo ip link set can0 down`	로봇 PC
운영 모드 켜기 / 끄기	`sudo systemctl start` / `stop camtic-robot-nav camtic-robot-zenoh`	로봇 PC
로봇 끄기	`sudo poweroff`	로봇 PC

확인

하고 싶은 것	명령어	위치
CAN 상태	`ip -br link show can0`	어디서나
바퀴 데이터	`ros2 topic hz /wheel_odom`	camtic_dev 안
베이스 상태	`ros2 topic echo --once /scout_status`	camtic_dev 안
라이다	`ros2 topic hz /scan`	camtic_dev 안
컨테이너 상태	`docker ps`	로봇 PC
CAN 자동 복구 기록	`journalctl -t can0-watchdog -f`	로봇 PC

개발

하고 싶은 것	명령어	위치
새 작업 폴더 만들기	`mkdir -p /home/camtic/my_ws/src`	어디서나
빌드 · 적용	`cd /home/camtic/my_ws && colcon build && source install/setup.bash`	camtic_dev 안

11. 주의사항

11-1. 안전

- 바퀴를 움직이는 시험은 로봇 주변에 사람과 장애물이 없는 넓은 곳에서 진행합니다.
- 처음 실행하거나 설정을 바꾼 뒤에는 바퀴가 바닥에 닿지 않게 들어 올린 상태에서 먼저 확인하는 것을 권장합니다.
- 배터리가 부족하면 바퀴 동작이 불안정할 수 있습니다. 충분히 충전한 상태에서 사용합니다.
- 전원은 `sudo poweroff`로 끕니다. 전원 버튼을 길게 눌러 강제로 끄면 파일이 손상될 수 있습니다.

11-2. 실행

- **개발 모드와 운영 모드를 동시에 실행하지 않습니다.** 바퀴 · 센서 프로그램이 두 번 실행되어 서로 충돌합니다.
- 같은 프로그램(예: 바퀴 프로그램)을 두 번 실행하지 않습니다. 실행 전에 이미 실행 중인지 확인합니다 (5-2의 2단계).
- 운영 모드는 `systemctl` 명령으로만 켜고 끕니다 (5-3).
- `docker compose up -d`를 컨테이너 이름 없이 실행하지 않습니다. 모든 컨테이너가 한꺼번에 켜지고, 이후 전원을 켤 때마다 자동으로 실행됩니다.
- 프로그램을 끌 때는 `Ctrl + C` 또는 이 문서에 적힌 명령만 사용합니다.

11-3. 시스템 설정

- 기존 작업 폴더 `camtic_ws` 안의 프로그램은 고치지 않습니다. 운영 모드와 CAN 자동 복구 기능이 이 폴더를 그대로 사용하므로, 고치면 기존 기능이 동작하지 않을 수 있습니다. 새 개발은 새 작업 폴더에서 합니다 (8-2).
- 다음 파일과 설정은 바꾸지 않습니다. 바꾸면 로봇이 부팅 후 정상적으로 동작하지 않을 수 있습니다.
  - `/home/camtic/camtic_ws/docker-compose.yml` (컨테이너 설정)
  - `/home/camtic/camtic_ws/cyclonedds.xml` (ROS 통신 설정)
  - `/etc/systemd/system/` 아래의 `camtic-robot*`, `can0-*` 서비스 (부팅 시 자동 실행)
  - 유선 고정 주소 `Backup` 프로필 (랜선 접속 주소)
- 랜선 직결 시 내 PC 주소를 로봇과 같은 `192.168.1.211`로 설정하지 않습니다. 주소가 겹쳐 접속되지 않습니다.

11-4. 개발

- camtic_dev 안에서 `apt install`, `pip install`로 설치한 프로그램은 컨테이너를 새로 만들면 사라집니다. 계속 쓸 프로그램은 8-4의 방법으로 추가합니다.
- camtic_dev를 다시 시작하거나 새로 만든 뒤에는 바퀴 프로그램이 자동으로 실행되지 않습니다. 필요하면 다시 실행합니다 (7-2).
- 작업한 파일은 `/home/camtic` 아래에 저장합니다. camtic_dev 안의 다른 위치(예: `/tmp`, `/opt`)에 저장한 파일은 컨테이너를 새로 만들면 사라집니다.